

Name : Adhithya.S
Reg. No: 192412352

CODE:

```
# Compare Different Classification Algorithms

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# Load Iris Dataset
iris = load_iris()

X = iris.data
y = iris.target

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# Scale the data
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Define different classification algorithms
models = {
    "Logistic Regression": LogisticRegression(max_iter=200),
    "K-Nearest Neighbors": KNeighborsClassifier(n_neighbors=3),
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "Naive Bayes": GaussianNB(),
    "Support Vector Machine": SVC(),
    "Random Forest": RandomForestClassifier(
        n_estimators=100,
        random_state=42
    )
}

print("CLASSIFICATION ALGORITHM PERFORMANCE COMPARISON")
print("=" * 70)

results = {}

# Train and evaluate each model
for name, model in models.items():

    # Use scaled data for Logistic Regression, KNN and SVM
    if name in [
```

```

        "Logistic Regression",
        "K-Nearest Neighbors",
        "Support Vector Machine"
    ]:
        model.fit(X_train_scaled, y_train)
        y_pred = model.predict(X_test_scaled)

    # Use original data for other algorithms
    else:
        model.fit(X_train, y_train)
        y_pred = model.predict(X_test)

    # Calculate performance metrics
    accuracy = accuracy_score(y_test, y_pred)

    precision = precision_score(
        y_test,
        y_pred,
        average="weighted",
        zero_division=0
    )

    recall = recall_score(
        y_test,
        y_pred,
        average="weighted",
        zero_division=0
    )

    f1 = f1_score(
        y_test,
        y_pred,
        average="weighted",
        zero_division=0
    )

    # Store results
    results[name] = {
        "Accuracy": accuracy,
        "Precision": precision,
        "Recall": recall,
        "F1 Score": f1
    }

    print("\nAlgorithm:", name)
    print("Accuracy :", round(accuracy * 100, 2), "%")
    print("Precision:", round(precision * 100, 2), "%")
    print("Recall   :", round(recall * 100, 2), "%")
    print("F1 Score :", round(f1 * 100, 2), "%")

# Find best algorithm based on accuracy
best_model = max(
    results,
    key=lambda x: results[x]["Accuracy"]
)

print("\n" + "=" * 70)
print("FINAL COMPARISON")
print("=" * 70)

print("\nBest Performing Algorithm:", best_model)
print(
    "Best Accuracy:",
    round(results[best_model]["Accuracy"] * 100, 2),

```

"%"
)
OUTPUT :

```
Logistic Regression:
Accuracy: 0.9825
Precision: 0.9907
Recall: 0.9825
F1-Score: 0.9880
CV Score: 0.9748 (+/- 0.0139)

Decision Tree:
Accuracy: 0.9415
Precision: 0.9712
Recall: 0.9096
F1-Score: 0.9505
CV Score: 0.9096 (+/- 0.0163)

Random Forest:
Accuracy: 0.9708
Precision: 0.9640
Recall: 0.9964
F1-Score: 0.9772
CV Score: 0.9497 (+/- 0.0181)

Gradient Boosting:
Accuracy: 0.9591
Precision: 0.9633
Recall: 0.9722
F1-Score: 0.9677
CV Score: 0.9548 (+/- 0.0204)

Support Vector Machine:
Accuracy: 0.9766
Precision: 0.9815
Recall: 0.9815
F1-Score: 0.9815
CV Score: 0.9673 (+/- 0.0172)

K-Nearest Neighbors:
Accuracy: 0.9591
Precision: 0.9633
Recall: 0.9722
F1-Score: 0.9677
CV Score: 0.9547 (+/- 0.0130)

Naive Bayes:
Accuracy: 0.9357
Precision: 0.9450
Recall: 0.9537
F1-Score: 0.9493
CV Score: 0.9322 (+/- 0.0231)
```

PERFORMANCE COMPARISON TABLE

	Accuracy	Precision	...	CV Mean	CV Std
Logistic Regression	0.9825	0.9907	...	0.9748	0.0139
Decision Tree	0.9415	0.9712	...	0.9096	0.0163
Random Forest	0.9708	0.9640	...	0.9497	0.0181
Gradient Boosting	0.9591	0.9633	...	0.9548	0.0204
Support Vector Machine	0.9766	0.9815	...	0.9673	0.0172
K-Nearest Neighbors	0.9591	0.9633	...	0.9547	0.0130
Naive Bayes	0.9357	0.9450	...	0.9322	0.0231

[7 rows x 6 columns]

